



SAPIENZA
UNIVERSITÀ DI ROMA

Impossibility of Automated Theorem Proving for Propositional Proof Systems by Analyzing Proofs of Meta-Complexity Statements

Faculty of Information Engineering, Computer Science and Statistics
Master's Degree in Computer Science

Simone Bianco

ID number 1986936

Advisor

Prof. Nicola Galesi

Prof. Massimo Lauria

Academic Year 2025/2026

**Impossibility of Automated Theorem Proving for Propositional Proof Systems
by Analyzing Proofs of Meta-Complexity Statements**

Master's Thesis. Sapienza University of Rome

© 2026 Simone Bianco. All rights reserved

This thesis has been typeset by $\text{\LaTeX}$ and the Sapthesis class.

Author's email: bianco.simone@outlook.it

something said by someone smarter than me
— not me

Abstract

Contents

Chapter 1

Introduction

1.1 Historical background

maybe move to automatability chapter

It's 1956. The genius mathematician John von Neumann is hospitalized at the Walter Reed Army Medical Hospital in Washington, USA. One year earlier, a mass was found near his collarbone, which turned out to be cancer, probably caused by exposure to radiation at the Los Alamos National Laboratory during World War II [Pou92]. After his condition improved thanks to a successful treatment, his colleague and friend Kurt Gödel sent a letter to express solidarity for his diagnosis, hoping for a full recovery. To keep von Neumann busy during his hospital stay, Gödel proposed his friend a seminal problem of fundamental interest both for logic and computer science.

«One can obviously easily construct a Turing machine, which for every formula F in first order predicate logic and every natural number n , allows one to decide if there is a proof of F of length n (length = number of symbols). Let $\psi(F, n)$ be the number of steps the machine requires for this and let $\varphi(n) = \max_F \psi(F, n)$. The question is how fast $\varphi(n)$ grows for an optimal machine. One can show that $\varphi(n) \geq kn$.

If there really were a machine with $\varphi(n) \sim kn$ (or even $\sim kn^2$), this would have consequences of the greatest importance. Namely, it would obviously mean that in spite of the undecidability of the Entscheidungsproblem, the mental work of a mathematician concerning Yes-or-No questions could be completely replaced by a machine. After all, one would simply have to choose the natural number n so large that when the machine does not deliver a result, it makes no sense to think more about the problem.»

Gödel to von Neumann, Princeton, 20 March 1956 [Sip92]

In a less formal way, Gödel's question asks if there is an efficient algorithm capable of deciding if a given mathematical statement has a proof of some given length. In modern times, the above problem became known as the *Proof Size Problem* (PSP). Sadly, an answer to Gödel's letter was never found and is supposed to have never existed. In fact, during his long hospital stay, von Neumann's mental health

rapidly declined due to his extreme thanatophobia, the fear of impending death. Almost a year later after being hospitalized, von Neumann died [Pou92].

In retrospective, Gödel's question is an historic precursor of the P vs. NP question. The original formulation of this latter question asks whether problems whose solutions are easy to double-check, i.e. problems who lay in the class NP , are also intrinsically easy to solve from scratch, i.e. if they also lay in the class P . The most interesting problems of these classes are the so called NP -complete problems, that being problems in NP whose efficient solution would imply an efficient solution for all NP problems, i.e. $P = NP$. The current state of the art conjectures that $P \neq NP$, meaning that these so-called complete problems cannot be efficiently solved in their general case.

If this were the case, there would be no better algorithmic approach than that of *brute force*, an approach that tests every candidate solution in the search space defined by the problem itself. For instance, if a problem asks to determine if there is a sub-structure with a specific property, the brute force approach tests every possible sub-structure. In his letter, Gödel himself states that he sees no better approach than that of reducing the search space through clever techniques.

During the years, researchers began to explore the Proof Size Problem from a restricted point of view. In particular they tried to prove or disprove the possibility of efficiently solving the problem for specific *proof systems*, i.e. logical systems that use restricted rules to find proofs of statements. For many weak proof systems it has been proven that the answer is negative, but it remains open whether the result may hold for more powerful proof systems.

add citation from
Kazuo Iwama

A more general question related to proof search and proof systems is that of *automatability*. Automatability asks if for a fixed proof system Q one can find an efficient algorithm that returns a Q -proof of a given statement, assuming it exists. Differently from the Proof Size Problem, which basically asks if short proof exists, automatability asks if short proofs can be constructed. The automatability of a proof system gives a way to approximate the minimum proof-size of a formula up to a polynomial factor (see).

add section on why
this is true

A central discovery of modern proof complexity is that many natural proof systems are not automatable under some valid complexity assumption (e.g. standard cryptographic assumptions or even $P \neq NP$). These results highlight a certain controversy in the world of proof systems. One would expect that a more expressive proof system would be capable of achieving remarkably short proofs, making it easier to generate proofs. However, the very same increase in expressive power implies that the search space of those proofs may be so unstructured that no algorithm can find them faster than brute force. In other words, it seems to be that if an algorithm fails to discover a short proof that is guaranteed to exist then the fault lies not in the algorithm, but in the proof space itself.

Add paragraph on
analyzability

Chapter 2

Preliminaries

2.1 Propositional logic

Propositional logic serves as the foundational mathematical framework for reasoning over declarative statements using boolean truth values and formal logical operators. For our purposes, we'll be interested in a fragment of propositional logic that works with finite amount of elements.

Let $\mathcal{P} = \{x_1, \dots, x_n\}$ denote a non-empty, finite set of atomic propositional symbols, called *variables*. Let $\Omega = \{\wedge, \vee, \neg, \rightarrow, \leftrightarrow\}$ be a set of logical connectives. Each of these constructors represents, respectively, the concept of conjunction, disjunction, negation, implication and logical equivalence.

The set of formulas $\text{Fml}(\mathcal{P})$ generated by $\mathcal{P}$ is the free algebra generated by $\mathcal{P}$. Inductively, $\text{Fml}(\mathcal{P})$ is the smallest set satisfying the following three conditions:

- $\mathcal{P} \subset \text{Fml}(\mathcal{P})$;
- If $\varphi \in \text{Fml}(\mathcal{P})$ then $\neg\varphi \in \text{Fml}(\mathcal{P})$;
- If $\varphi, \psi \in \text{Fml}(\mathcal{P})$ and $\circ \in \Omega$ then $(\varphi \circ \psi) \in \text{Fml}(\mathcal{P})$.

The set $\text{Fml}(\mathcal{P})$ defines propositional logic from a syntactical point of view, i.e. how correct formulas are written. To give meaning to the formulas, we have to also define it from a semantical prospective. An *assignment* is a mapping $\alpha : \mathcal{P} \rightarrow \{0, 1\}$ that sets a truth value to each variable. By extension, each assignment α over variables induces an assignment $\hat{\alpha} : \text{Fml}(\mathcal{P}) \rightarrow \{0, 1\}$ through the semantical meaning of the logical connectives:

- $\hat{\alpha}(\neg\varphi) = 1$ if and only if $\hat{\alpha}(\varphi) = 0$
- $\hat{\alpha}(\varphi \wedge \psi) = 1$ if and only if $\hat{\alpha}(\varphi) = \hat{\alpha}(\psi) = 1$
- $\hat{\alpha}(\varphi \vee \psi) = 0$ if and only if $\hat{\alpha}(\varphi) = \hat{\alpha}(\psi) = 0$
- $\hat{\alpha}(\varphi \rightarrow \psi) = 1$ if and only if $\hat{\alpha}(\psi) = 1$ whenever $\hat{\alpha}(\varphi) = 1$ holds

φ	ψ	$\neg\varphi$	$\varphi \wedge \psi$	$\varphi \vee \psi$	$\varphi \rightarrow \psi$	$\varphi \leftrightarrow \psi$
1	1	0	1	1	1	1
1	0	0	0	1	0	0
0	1	1	0	1	1	0
0	0	1	0	0	1	1

Table 2.1. Summary of truth assignments for logical connectives in Ω .

- $\hat{\alpha}(\varphi \leftrightarrow \psi) = 1$ if and only if $\hat{\alpha}(\varphi \rightarrow \psi) = \hat{\alpha}(\psi \rightarrow \varphi) = 1$

By recursively combining these constructs, we can build more complex formulas that fully express a sentence. For instance, the sentence “I’ll take my umbrella and my boots if it rains” can be modeled by the formula $R \rightarrow (U \wedge B)$. A formula $\varphi \in \text{Fml}(\mathcal{P})$ is said to be *satisfiable* if there is at least one assignment α such that $\alpha(\varphi) = 1$. When all possible assignments satisfy a formula, we refer to that formula as a *tautology*, or simply as true formulas. Clearly, φ is a tautology if and only if $\neg\varphi$ is unsatisfiable.

Given a Boolean variable x , a *literal* ℓ of x is either the variable x_i itself or its negation $\neg x$. A *clause* is a disjunction of literals, $C = \ell_1 \vee \dots \vee \ell_k$, while a *term* is a conjunction of them, $T = \ell_1 \wedge \dots \wedge \ell_t$. A *conjunctive normal form* (CNF) formula φ is a conjunction of m clauses $C_1, \dots, C_m$. Similarly, a *disjunctive normal form* (DNF) formula is a disjunction of s terms $T_1, \dots, T_s$. Any formula can be written as an equivalent CNF or an equivalent DNF. For instance, the formula $\varphi = ((x_1 \vee \neg x_2) \wedge x_3) \vee x_4$ can be written as the CNF formula $(x_1 \vee \neg x_2 \vee x_4) \wedge (x_3 \vee x_4)$ or as the DNF formula $(x_1 \wedge x_3) \vee (\neg x_2 \wedge x_3) \vee x_4$.

2.2 Encodings of combinatorial statements (TODO)

2.3 Computability

Depending on the objective, a problem can ask for a binary answer, a specific solution structure or the best possible solution out of many. We divide these problems into three categories: *decision*, *search* and *optimization*. A decision problem is a problem for which we’re asked to determine if a chosen property holds for a given an instance of the problem. A search problem, instead, asks to find some structure that certifies the validity of the chosen property. Similarly, an optimization problem asks us to find the optimal structure among all the available ones. It’s easy to see that these three ideas form some sort of hierarchy between types of problems. In general, optimization problems are harder than search problems, which in turn are harder than decision problems.

We face these types of problems every day in our life. For instance, suppose that we want to reach a destination B from a starting point A . If we’re interested in finding the *shortest* route between point A and B , this becomes an optimization

problem. If we want find any kind of route between the two sites, without caring about its length, this becomes a search problem. If, instead, we want to just establish the existence of at least one route, this becomes a decision problem.

Let Σ be a finite set of symbols (e.g. $\Sigma = \{a, b, c, 0, \$, 3, x, \dots\}$), referred to as *alphabet*, and let Σ^* be the infinite set of all finite strings formed by characters taken from that alphabet. Any subset $L \subseteq \Sigma^*$ is referred to as a *language*. Typically, we're interested to the case where $\Sigma = \{0, 1\}$.

Consider a binary predicate $P : \Sigma^* \rightarrow \{0, 1\}$ representing some property, meaning that $P(x) = 1$ means that the property holds for the input instance x and $P(x) = 0$ means that it doesn't. Let $R_P \subseteq \Sigma^* \times \Sigma^*$ be the binary relation such that $R_P(x, y)$ holds if and only if y certifies the fact that $P(x) = 1$. We refer to $\text{Sol}_P(x) = \{y \in \Sigma^* \mid R_P(x, y)\}$ as the set of *solutions* (or *witnesses*) for the instance x over the property P .

Definition 2.1. The decision problem of P is defined as the language $\Pi_{\text{dec}}^P = \{x \in \Sigma^* \mid \text{Sol}_P(x) \neq \emptyset\}$. The problem Π_{dec}^P is said to be computable if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) = 1$ when $\text{Sol}_P(x) \neq \emptyset$ and $\mathcal{A}(x) = 0$ otherwise.

We observe that the above definition of computability for decision problem can also be stated as the existence of an always-halting algorithm $L(\mathcal{A}) = \Pi_{\text{dec}}^P$. Moreover, we also observe that the above definition could've been stated without the use of the binary relationship R_P since $\Pi_{\text{dec}}^P = \{x \in \Sigma^* \mid P(x) = 1\}$ is an equivalent definition. In this setting, the definition is given using R_P only for consistency with search and optimization problems. In fact, for the other two types of problems binary relationships are necessary due to the fact that one instance may have more than one possible witness. For instance, there may be more than one (optimal) route from point A to point B .

Definition 2.2. The search problem of P is defined as the binary relationship $\Pi_{\text{search}}^P = R_P$. The problem Π_{search}^P is said to be computable if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) \in \text{Sol}_P(x)$ when $\text{Sol}_P(x) \neq \emptyset$ and $\mathcal{A}(x) = \perp$ otherwise.

Here, the symbol $\perp$ is used to represent the fact that no solution exists for the given input instance. We observe that the definition allows any algorithm computing the problem to return a different solution on each execution. Optimization problems require an additional component: a *cost function* $c : \Sigma^* \times \Sigma^* \rightarrow \mathbb{R}$ expressing the optimality of the solution for the given instance. We denote the set of optimal solutions as $\text{Opt}_P(x) = \{y \in \text{Sol}_P(x) \mid c(x, y) = \max_{z \in \text{Sol}_P(x)} c(x, z)\}$.

Definition 2.3. The optimization problem of P over c is defined as the binary relationship $\Pi_{\text{opt}}^P = \{(x, y^*) \in \Sigma^* \times \Sigma^* \mid y^* \in \text{Opt}_P(x)\}$. The problem Π_{opt}^P is said to be computable if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) \in \text{Opt}_P(x)$ when $\text{Sol}_P(x) \neq \emptyset$ and $\mathcal{A}(x) = \perp$ otherwise.

As previously stated, these three categories form an hierarchy of problems. This comes clear from their own definitions: any algorithm computing an optimization problem also computes a search problem and any algorithm computing a search problem can be adapted into one that computes a decision problem.

2.4 Computational complexity

In his seminal work, Turing [Tur37] proved that some problems are *undecidable*, meaning that there is no algorithm that can compute them exactly. Following his work, the pioneers of computer science started noticing that even though many problems can be computed, the algorithms computing them would take too much resources, making them infeasible to process for large inputs. In particular, they focused on two main measures: *time* and *space*.

Definition 2.4. Let $\mathcal{A}$ be an algorithm and let $t, s: \mathbb{N} \rightarrow \mathbb{R}^+$ be two functions. We say that $\mathcal{A}$ *runs in time* t if on every input x the algorithm halts after at most $t(|x|)$ steps and that $\mathcal{A}$ *runs in space* s if on every input x it uses at most $s(|x|)$ cells memory cells.

These two complexity measures are intrinsically related one to another and they are, usually, negatively correlated. Generally, if we allow our algorithm to use more space then the computation can be sped up by saving partial computations, while if we want to lower the amount of space needed then these partial computation have to be computed each time they're required.. Larger inputs require a larger amount of computational resources, making the values $t(n)$ and $s(n)$ proportional to the size n of the input. For this reason, as the input size grows, we are interested in the asymptotic behavior of these measures.

To distinguish between efficiently solvable problems and intractable ones, we usually consider the class P . This class is defined as the set of all decision problems solvable in polynomial time by an algorithm, meaning that there is an algorithm that computer the problem in time $O(n^c)$, for some $c > 0$. Similarly, we consider the class FP as the class of search problems solvable in polynomial time by an algorithm.

We observe that this definition allows the exponent c to be as huge as possible. In real world situations, any algorithm with runtime larger than $O(n^3)$ is considered useless since it requires too much time even for smaller inputs. Nonetheless, this can be used at our advantage: if a problem doesn't lie in P , it means that there is no algorithm for it even if we were to consider infinitely large values of c . Therefore, a decision problem L is considered efficiently solvable when it lies in P , while it is considered intractable when it doesn't. The same reasoning applies to the class FP .

The aforementioned P vs. NP (or the FP vs. FNP question for search problems) question asks to settle if the class of efficiently solvable problems corresponds to the class of efficiently verifiable problems. Here, a *verifier* for a problem L is an algorithm that takes an additional input w , referred to as the *witness*, and uses it to check if $x \in L$ holds or not. Usually, this witness is a string that describes a real solution of the problem for the given input. By definition, we have that $P \subseteq NP$ since if a problem is efficiently solvable then we can just ignore the given witness and solve the problem. Hence, the P vs. NP question asks to settle if this inclusion also holds in the converse direction, i.e. $NP \subseteq P$. After almost 60 years of scientific literature studying this problem and its ramifications, researchers conjecture that the answer to this problem is negative.

Conjecture 2.1. $P \neq NP$

For our purposes, we'll assume this conjecture to be true. Another class of our interest is the class **coNP**, defined as the set of all problems whose complement lies in **NP**, i.e. $\text{coNP} = \{L \subseteq \Sigma^* \mid \bar{L} \in \text{NP}\}$. It's easy to see that problems that lie in **coNP** admit a verifier that use witnesses to discard invalid input instances: if w is a witness for the membership of x in $\bar{L}$ then w is also a witness for the non-membership of x in L . In other words, a problem is in **NP** if its correct instances can be efficiently verified, while a problem is in **coNP** if the same holds for incorrect instances.

Again, it's easy to see that $P \subseteq \text{coNP}$, but the converse is not known and it is actually a question equivalent to establishing that $P \subseteq \text{NP}$. This is due to the fact that if a problem is efficiently solvable then its complement is also efficiently solvable (we can just negate the output of the same algorithm). This same reasoning implies that if $P = \text{NP}$ then $\text{NP} = \text{coNP}$. Therefore, proving that $\text{NP} \neq \text{coNP}$ could be a way of establishing that $P \neq \text{NP}$. It is also conjectured that $\text{NP} \neq \text{coNP}$ since there are evident asymmetries between the hardness of witnessing that something holds and that of witnessing something everywhere in mathematics and computer science.

Conjecture 2.2. $\text{NP} \neq \text{coNP}$

Finally, we introduce the concept of reductions between problems. Let A and B be two decision problems. We say that A *many-to-one reduces* to B , written as $A \leq_m B$ when there is a computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that $x \in A$ if and only if $f(x) \in B$. The idea of problem reduction is simple: if a reducing function exists and the new problem can be solved then the original problem can also be solved. This is due to the fact that if an algorithm for the original problem has to simply compute the reduction and then execute the algorithm that solves the new problem.

It's easy to see that the relation $\leq_m$ forms a partial order over the power set $2^{\Sigma^*} = \{S \subseteq \Sigma^*\}$, corresponding to the set of all decision problems. When $A \leq_m B$ holds, we say that B is “at least as hard as” A since a solving B implies solving A . If the reduction is computable in polynomial time, we say that A *p-reduces* to B , written as $A \leq_p B$. In two seminal papers, Cook and Levin independently showed that every problem that lies in **NP** is reducible to the *satisfiability problem* $\text{SAT} = \{\langle \varphi \rangle \mid \varphi \text{ a satisfiable CNF}\}$, where $\langle \cdot \rangle$ is the binary encoding. Problems with this property are referred to as **NP-hard** and, under the assumption that $P \neq \text{NP}$, they cannot be solved in polynomial time. Therefore, showing the hardness of a problem comes as a tool of showing that it cannot be efficiently solved.

[add reference](#)

2.5 Decision trees

Decision trees are a common simple computational model used in many areas of Computer Science. In our case, we'll consider decision trees as a way to compute Boolean functions $f : \{0, 1\}^n \rightarrow \{0, 1\}$ or, more generally, for search problems based on such functions. Informally, decision trees are graphical representations of algorithms for computing a function of an unknown input. Each node of the tree is labeled by a variable x_i , $i \in [n]$, and the branches from that node are labeled by the possible values of the variable (0 or 1 in our case). The leaves are labeled by

the output of the function. The process starts at the root, knowing nothing, works down the tree, choosing to learn the values of some of the variables based on those already known and eventually reaches a decision.

Definition 2.5 ([search_problems_dt_model]). A decision tree is a rooted directed binary tree whose nodes are associated with either an output value or an input Boolean variable. Each internal node is labeled by a variable and the two outgoing edges are labeled by the two possible values of that variable. Each leaf is labeled with an output $b \in \{0, 1\}$.

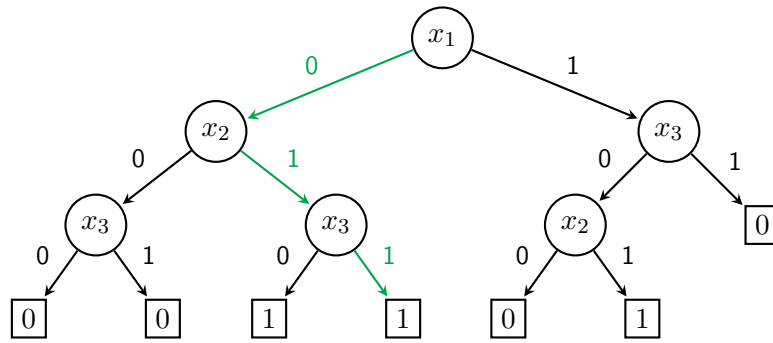


Figure 1. An example of a decision tree of size 13 and depth 3. The green path shows the computation made for the input $x = 011$.

A Boolean function f is said to be computed by a decision tree $\mathcal{T}$ when for each input x it holds that $f(x) = T(x)$. The *depth* of a decision tree is the number of edges in a longest path from the root to a leaf, or equivalently, the maximum number of bits tested on such a path. Generally, for a Boolean function f we denote with $D(f)$ the minimum depth across all decision trees computing f .

An interesting property of decision trees is their possibility to be represented as propositional formulas. In particular, a depth- d decision tree $\mathcal{T}$ computing f gives rise to both a d -DNF and a d -CNF representation for f .

Let $\text{Leaves}(\mathcal{T})$ be the set of all leaves in $\mathcal{T}$ and let $\text{out}(\ell)$ be the output value associated with a leaf $\ell \in \mathcal{T}$. Then, the associated d -DNF $\text{DNF}(\mathcal{T})$ is given by

$$\text{DNF}(\mathcal{T}) := \bigvee_{\substack{\ell \in \text{Leaves}(\mathcal{T}): \\ \text{out}(\ell)=1}} C_\ell$$

where C_ℓ is the conjunction of all the literals corresponding to the query outcomes on the path from the root of $\mathcal{T}$ to the leaf ℓ . If the query x_i has outcome 1, the corresponding literal is x_i , otherwise it is $\neg x_i$. The associated d -CNF, instead, is obtained by negating the d -DNF formula associated with the negated decision tree $\neg\mathcal{T}$, corresponding to the tree $\mathcal{T}$ with inverted output values, computing $\neg f$. For instance, the decision tree shown in ?? can be represented as the DNF formula

$$(\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge x_3) \vee (x_1 \wedge \neg x_3 \wedge x_2)$$

Chapter 3

Proof complexity

3.1 Propositional proof systems

Proof complexity is the branch of computational complexity and logic that studies the resources that are needed in order to *certify* mathematical statements, rather than of the resources needed to *decide* them. At its core, proof complexity asks if there is a way of writing down proofs of propositional tautologies so that every tautology has short proofs. Such ways of proof writing can be viewed as sets of rules capable of manipulating propositional formulas, deriving new formulas from them. These sets of rules are referred to as *propositional proof systems*. When a formula ψ can be derived from a formula φ through the rules of a proof system Q , we write $\varphi \stackrel{Q}{\vdash} \psi$ or simply $\varphi \vdash \psi$ when the context is clear. The easiest example of propositional proof system is *Resolution*, based on two single rules:

$$\frac{C \vee x \quad D \vee \neg x}{C \vee D} \quad (\text{resolution}) \qquad \frac{C}{C \vee B} \quad (\text{weakening})$$

For a more detailed discussion on examples of proof systems, see the next section . In the seminal paper from which proof complexity originated, defined proof systems as surjective functions $Q : \Sigma^* \rightarrow \text{TAUT}$, where the latter is the set of all tautologies, $\text{TAUT} = \{\langle \varphi \rangle \mid \varphi \text{ a CNF tautology}\}$. The idea behind this definition is that of an algorithm that is able to determine which tautology is proven by a given proof. A modern and more descriptive definition can be achieved through the concept of verifier.

add ref

Definition 3.1. A propositional proof system (p.p.s.) is a verifier Q such that for all formulas φ it holds that $\varphi \in \text{TAUT}$ if and only if there is a proof $\pi \in \Sigma^*$ such that $Q(\varphi, \pi) = 1$ and Q runs in time $O((|\varphi| + |\pi|)^c)$ for some $c > 0$.

This if and only if definition can be split in two implications called completeness and soundness. A proof system Q is *complete* if whenever a formula has a Q -proof it also is a tautology. Vice versa, Q is *sound* if whenever a formula is a tautology it also has a Q -proof. When π is a Q -proof of φ , we often write $\pi : Q \vdash \varphi$.

When for each tautology there is a short proof, i.e. $|\pi| = O(|\varphi|^c)$ for some $c > 0$, we say that the proof system is *polynomially bounded*. Proof systems were originally

introduced by Cook and Reckhow as a possible way of attacking the NP vs. coNP problem. In fact, through the modern definition, it's immediate to see that finding a polynomially bounded p.p.s. is equivalent to proving that $\text{NP} = \text{coNP}$.

Proposition 3.1. *There is a polynomially bounded p.p.s. if and only if $\text{NP} = \text{coNP}$*

Proof. One can easily prove that TAUT is coNP-complete. For our purposes, we can omit this proof and assume it to hold. Let Q be a polynomially bounded p.p.s. Then, since for each formula there is a proof of size $|\pi| = O(|\varphi|^c)$ for some $c > 0$, Q is a verifier for TAUT with polynomial runtime with respect to the input size $|\varphi|$, concluding that $\text{TAUT} \in \text{NP}$. By completeness, we get that $\text{NP} = \text{coNP}$.

Vice versa, suppose that $\text{NP} = \text{coNP}$. Then, since $\text{TAUT} \in \text{NP}$, there is a polynomial time verifier V for TAUT. In order for it to run in polynomial time, for each formula φ there must be at least one witness t such that $|w| = O(|\varphi|^d)$ for some $d > 0$, otherwise reading the witness would always require super-polynomial runtime, raising a contradiction. This concludes that V is a polynomially bounded p.p.s. $\square$

In their programme, Cook and Reckhow outlined a possible roadmap to proving that $\text{NP} \neq \text{coNP}$ through p.p.s. Their idea involved proving that strong proof systems require super-polynomial size proofs for some formulas, to then generalize the various results to any possible proof system, i.e. showing that every proof system has at least one formula that is hard to prove. However, researchers quickly realized that this task would be tedious even for weaker systems. In fact, even after almost 40 years of research in the field, only a few weak systems have been completely outlined (e.g. Resolution).

The programme has also a second, entirely practical, motivation. Every complete algorithm that established the satisfiability of a propositional formula, referred to as a *SAT-solver*, implicitly produces a proof of that formula in some proof system. For instance, the DPLL algorithm (Davis-Putnam-Logemann-Loveland) and CDCL solvers (Conflict-driven Clause Learning) produce resolution proofs, while Gröbner basis solvers produce algebraic proofs. A lower bound on proof size in a system is therefore an unconditional lower bound on the running time of *every* algorithm of the corresponding kind, regardless of how cleverly it is implemented. Proof complexity is, from this point of view, the theory of the limits of automated reasoning.

3.2 Common proof systems

3.2.1 Resolution

The Resolution proof system (Res), introduced by Robinson, is the most extensively studied proof system, extensively in automated theorem proving, SAT-solvers and artificial intelligence. Like many weak proof systems, Resolution's proofs are actually *refutations*, i.e. “proofs” of unsatisfiability of a formula. This is a feature, not an issue: since φ is a tautology if and only if $\neg\varphi$ is unsatisfiable, any refutation of $\neg\varphi$ is actually a proof of φ . In many works, the word refutation is often used as a synonym of proof. To help the reader, this won't be the case in this work.

add ref

$D_1 :$	$\neg x_3$	Axiom
$D_2 :$	$\neg x_1 \vee \neg x_2 \vee x_3$	Axiom
$D_3 :$	$x_1 \vee \neg x_2 \vee x_3$	Axiom
$D_4 :$	$\neg x_1 \vee \neg x_2$	$\text{Res}(D_1, D_2)$
$D_5 :$	$x_1 \vee \neg x_2$	$\text{Res}(D_1, D_3)$
$D_6 :$	$x_2 \vee x_3$	Axiom
$D_7 :$	$x_2 \vee \neg x_3$	Axiom
$D_8 :$	x_2	$\text{Res}(D_6, D_7)$
$D_9 :$	$\neg x_1$	$\text{Res}(D_4, D_8)$
$D_{10} :$	x_1	$\text{Res}(D_5, D_8)$
$D_{11} :$	$\perp$	$\text{Res}(D_9, D_{10})$

Figure 2. Example of a Resolution refutation for the formula $(x_2 \vee x_3) \wedge (x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge \neg x_3$

Resolution refutations are based on repeated applications of two simple rules, starting from the axioms of the initial formula:

$$\frac{C \vee x \quad D \vee \neg x}{C \vee D} \quad (\text{resolution}) \qquad \frac{C}{C \vee B} \quad (\text{weakening})$$

Given a CNF formula $\varphi = C_1 \wedge \dots \wedge C_m$, let $\text{Ax}(\varphi) = \{C_1, \dots, C_m\}$ be the set of axioms of φ . Then, a Resolution refutation of φ is a sequence of clauses $\pi = (D_1, D_2, \dots, D_t)$ such that for each $i \in [t]$ exactly one of the following holds:

- *Axiom:* $D_i \in \text{Ax}(\varphi)$;
- *Resolution:* $D_i = \text{Res}(D_j, D_k)$ with $j, k \in [i-1]$;
- *Weakening:* $D_i = \text{Weak}(D_j)$ with $j \in [i-1]$.

For the last clause D_t , it must hold that $D_t = \perp$, where $\perp$ represents the *empty clause*, i.e. the clause that doesn't contain any literal.

The idea behind the Resolution proof system comes from the propagation of satisfiability underlying the resolution rule and the weakening rule.

Proposition 3.2. *Let α be an assignment. If $\alpha(C \vee x) = 1$ and $\alpha(D \vee \neg x) = 1$ then $\alpha(C \vee D) = 1$. Similarly, if $\alpha(C) = 1$ then $\alpha(C \vee B) = 1$*

One can easily prove by induction that if a clause Z is derived through repeated applications of resolution and weakening starting from the axioms then $\alpha(\varphi) = 1$ implies $\alpha(Z) = 1$. When $Z = \perp$, we reach a contradiction since, by definition, the empty clause is unsatisfiable. Therefore, if we're able to derive $\perp$ starting from $\text{Ax}(\varphi)$, we obtain a valid proof of the fact that φ must be unsatisfiable. This proves the soundness of Resolution and completeness can be proven in a similar way.

For any fixed natural number $k \in \mathbb{N}$, we define an extension of the Resolution proof system called *Resolution over k -DNFs*, denoted $\text{Res}(k)$, first introduced by . This proof system works over sets of k -DNF formulas, expressed as a disjunction of

conjunctions of at most k literals. Its rules are inspired by to those of Resolution, adapting them to manage k -DNFs:

$$\begin{array}{c}
 \frac{\mathcal{D}_1 \vee \left(\bigvee_{j \in J} \ell_j \right) \quad \mathcal{D}_2 \vee \left(\bigwedge_{j \in J} \neg \ell_j \right)}{\mathcal{D}_1 \vee \mathcal{D}_2} \quad (\text{cut}) \\
 \\
 \frac{\mathcal{D}_1 \vee \left(\bigwedge_{j \in J} \ell_j \right) \quad \mathcal{D}_2 \vee \left(\bigwedge_{j' \in J'} \ell_{j'} \right)}{\mathcal{D}_1 \vee \mathcal{D}_2 \vee \left(\bigvee_{j \in J \cup J'} \ell_j \right)} \quad (\wedge \text{ introduction}) \\
 \\
 \frac{\mathcal{D}_1}{\mathcal{D}_1 \vee \left(\bigwedge_{j \in J} \ell_j \right)} \quad \frac{\mathcal{D} \vee \left(\bigwedge_{j \in J \cup J'} \ell_j \right)}{\mathcal{D}_1 \vee \left(\bigvee_{j \in J} \ell_j \right)} \quad (\text{weakenings})
 \end{array}$$

where $\mathcal{D}_1$ and $\mathcal{D}_2$ are k -DNF formulas.

For standard Resolution, it can be proven that the weakening rule is actually not necessary. In particular, for each Resolution refutation π there is an equivalent refutation π' for the same formula φ such that π' never uses the weakening rule and such that π' uses at most the same number of steps of π . This property will be useful to manipulate refutations by making assumptions on how and when the weakening steps are applied inside them.

3.2.2 Frege systems

Frege systems are a more classical type of propositional proof system, whose proofs are intended to infer tautologies. These proof systems are well known in the classical Logic literature and, furthermore, they work not only with CNF formulas, but also with general Boolean formulas.

Formally, a Frege system is a propositional proof system built from a finite set of axiom schemes and a finite set of rules of inference that can be applied to sets of formulas. An *axiom scheme* is a meta-formula with variables over formulas that can be substituted by any formula. For instance, both $x_1 \wedge x_2 \rightarrow x - 1$ and $(x_1 \vee x_2) \wedge x_2 \rightarrow (x_1 \vee x_2)$ are formulas that can be obtained from the axiom schema $(P \wedge Q) \rightarrow Q$, where P and Q are variables over $\text{Fml}(\mathcal{P})$. A *rule scheme* is a typical rule of logical derivation. A prominent example of a schema rule is the *Modus Ponens* (MP), defined as:

$$\frac{P \quad P \rightarrow Q}{Q}$$

Modus Ponens allows to cut implications from formulas of the form $P \rightarrow Q$ once the formula P alone is derived. We can define different Frege systems introducing different axiom schemes. Nonetheless, Reckhow proved that all Frege systems are polynomially equivalent to each other, meaning that the axiom scheme and rule scheme chosen is not relevant, up to some polynomial factor (see Section for an explanation of polynomial equivalence). Therefore, we don't actually care about which rules and axioms are used in our Frege system. This allows us to restrict our interest to a standardized Frege system, known in the literature as *Hilbert's*

add ref

add ref

propositional proof system, denoted with F or Frege. The rule scheme of this proof system allows only the use of Modus Ponens over the following standard axiom scheme $\text{Ax}(\text{F})$ of Hilbert-style systems for propositional logic:

- | | | |
|------|---|---------------------|
| (1) | $A \rightarrow B \rightarrow (A \wedge B)$ | |
| (2) | $(A \wedge B) \rightarrow A$ | $\wedge$ rules |
| (3) | $(A \wedge B) \rightarrow B$ | |
| (4) | $A \rightarrow (A \vee B)$ | $\vee$ rules |
| (5) | $B \rightarrow (A \vee B)$ | |
| (6) | $(A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow ((A \vee B) \rightarrow C)$ | |
| (7) | $(\perp \rightarrow A)$ | $\neg$ rules |
| (8) | $(A \rightarrow \perp) \rightarrow \neg A$ | |
| (9) | $A \rightarrow (\neg A \rightarrow \perp)$ | |
| (10) | $(\neg A \rightarrow \perp) \rightarrow A$ | |
| (11) | $A \rightarrow A$ | $\rightarrow$ rules |
| (12) | $A \rightarrow (B \rightarrow A)$ | |
| (13) | $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$ | |

Given a formula A , an F-proof of A is sequence of formulas $\pi = (A_1, \dots, A_t)$ such that each $i \in [t]$ exactly one of the following holds:

- *Axiom*: $A_i \in \text{Ax}(\text{F})$;
- *Modus Ponens*: $A_i = \text{MP}(A_j, A_k)$ with $j, k \in [i - 1]$.

For the last formula A_t , it must hold that $A = A_t$. The soundness of F comes immediately from the soundness of Modus Ponens, something that assumed as an axiom in standard settings of classical Logic. Completeness follows similarly.

Extended Frege systems, denoted with EF, are Frege systems augmented with an *extension rule* that allows the introduction of new variables that abbreviate more complex formulas.

- *Extension*: $A_i = q \leftrightarrow B$ where q is a new variable that never appeared before and B is any propositional formula.

Intuitively, this rule allows us to write shorter proofs by encoding partial statements as single variables and use it in the proof, instead of carrying over the initial statement across the whole proof. In some sense, Frege proofs can be viewed as proofs that reason about polynomial size Boolean formulas, while Extended Frege proofs as proofs that reason about polynomial size Boolean circuits.

3.2.3 Algebraic systems

Algebraic proof systems are proof systems based on polynomials. These systems can be used to witness the fact that a system of polynomial equations over a the

polynomial ring $\mathbb{F}[x_1, \dots, x_n]$ is unsolvable; equivalently, that the polynomials do not have a common root.

Consider a CNF formula φ defined over $x_1, \dots, x_n$. The formula can be easily encoded as a set of polynomials such that the formula is unsatisfiable if and only if such set of polynomials has no solution. We map each clause of φ to an equation in the following way:

$$\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \mapsto \prod_{i \in I} x_i \prod_{j \in J} (1 - x_j) = 0$$

where I and J are, respectively, the set of positive and negative literals of the clause. For each clause C , its corresponding polynomial is denoted with $P(C)$. The formula φ is unsatisfiable if and only if the system $\Phi = \{P(C)\}_{C \in \varphi} \cup \{x_i^2 - x_i = 0\}_{i \in [n]}$ has no solutions. In particular, the latter set of equations is used to restrict all possible solutions to those that give Boolean values to $x_1, \dots, x_n$.

After this transformation is made, the question of whether a formula is unsatisfiable is reduced to the question of a set of polynomial equations is unsolvable. Algebraic tools can then be used in order to decide unsatisfiability of the original formulas. We notice that, under this framework, truth is inversely represented: the value 0 represents truth, while 1 represents falsity.

Two particular algebraic proof systems are extensively studied in proof complexity: Nullstellensatz and Polynomial Calculus. *Nullstellensatz* (NS) is a static algebraic proof system, i.e. one that doesn't use rules of inference, based on Hilbert's Nullstellensatz, which states that given m polynomials $p_1, \dots, p_m$ the system $p_1(x) = p_2(x) = \dots = p_m(x) = 0$ is unsolvable if and only if 1 lies in their generated ideal, i.e. there are m polynomials $g_1, \dots, g_m$ such that $\sum_{j=1}^m g_j p_j = 1$. For any CNF φ , an NS-proof of φ is the set of polynomials $\pi = \{g_1, \dots, g_m, h_1, \dots, h_m\}$ such that:

$$\sum_{j=1}^m g_j p_j + \sum_{i=1}^n h_i (x_i^2 - x_i) = 1$$

where $p_j = P(C_j)$ for each $j \in [m]$.

Polynomial Calculus (PC) is a dynamic proof system based on two rules:

$$\frac{p}{\alpha p + \beta q} \quad (\text{linear combination}) \qquad \frac{p}{xp} \quad (\text{multiplication})$$

where $\alpha, \beta \in \mathbb{F}$ and $x \in \{x_1, \dots, x_n\}$. A PC-proof of the CNF φ is a sequence of polynomials $\pi = (f_1, \dots, f_t)$ such that each $i \in [t-1]$ exactly one of the following holds:

- *Axiom*: f_i is the polynomial of some equation in Φ
- *Linear combination*: $f_i = \text{Sum}(f_j, f_k)$ with $j, k \in [i-1]$
- *Multiplication*: $f_i = \text{Mul}(f_j)$ with $j \in [i-1]$

For the last polynomial f_t , it must hold that $f_t = 1$. The soundness of PC can be proven in way similar to Resolution: if we assume that the system of equations Π has a solution, the equality with 0 can be propagated down to the last polynomial $f_t = 1$, raising a contradiction. Completeness follows similarly.

3.2.4 Geometric systems

Geometric proof systems are proof systems based on linear programming. All such proof systems are variants of the *Cutting Planes* (CP) proof system, a refutational method derived from 0-1 integer programming. In CP, any CNF formula φ is converted into an equivalent system of linear inequalities (similar to how algebraic systems convert them into equations) whose integer binary solutions correspond directly to the satisfying assignments of φ . Each clause of a formula φ is mapped to an inequality in the following way:

$$\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \mapsto \sum_{i \in I} x_i + \sum_{j \in J} (1 - x_j) \geq 1$$

For each clause C , its corresponding inequality is denoted with $E(C)$. Interpreting truthness as 1 and falseness as 0, the formula φ is unsatisfiable if and only if the system $\Phi = \{E(C)\}_{C \in \varphi} \cup \{x_i \geq 0; x_i \leq 1\}_{i \in [n]}$ has no an integer solution. The set of all feasible solutions for this system yield a polytope P . While any Boolean point in P yields a satisfying assignment for F , an unsatisfiable formula F may still leave non-integer (fractional) points within P . CP eliminates these fractional solutions using the *Chvátal-Gomory cut rule*. In particular, CP is based on the two following rules:

$$\frac{\sum_{i \in [n]} \alpha_i x_i \geq \gamma \quad \sum_{i \in [n]} \beta_i x_i \geq \delta}{\sum_{i \in [n]} (\alpha_i + \beta_i) x_i \geq \gamma + \delta} \quad (\text{linear combination})$$

$$\frac{\sum_{i \in [n]} \eta \alpha_i x_i \geq \gamma}{\sum_{i \in [n]} \alpha_i x_i \geq \left\lfloor \frac{\gamma}{\eta} \right\rfloor} \quad (\text{cut})$$

Formally, a CP-proof of the CNF φ is a sequence of inequalities $\pi = (f_1, \dots, f_t)$ such that each $i \in [t-1]$ exactly one of the following holds:

- *Axiom*: $f_i \in \Phi$
- *Linear combination*: $f_i = \text{Sum}(f_j, f_k)$ with $j, k \in [i-1]$
- *Multiplication*: $f_i = \text{Cut}(f_j)$ with $j \in [i-1]$

For the last inequality f_t , it must hold that $f_t : 0 \geq 1$ to ensure a contradiction.

3.2.5 Tree-like systems

Consider again the refutation in ???. To make things easier to read, we can represent this refutation as a Directed Acyclic Graph (DAG) by connecting clauses with lines. These refutations are called *DAG-like refutations*.

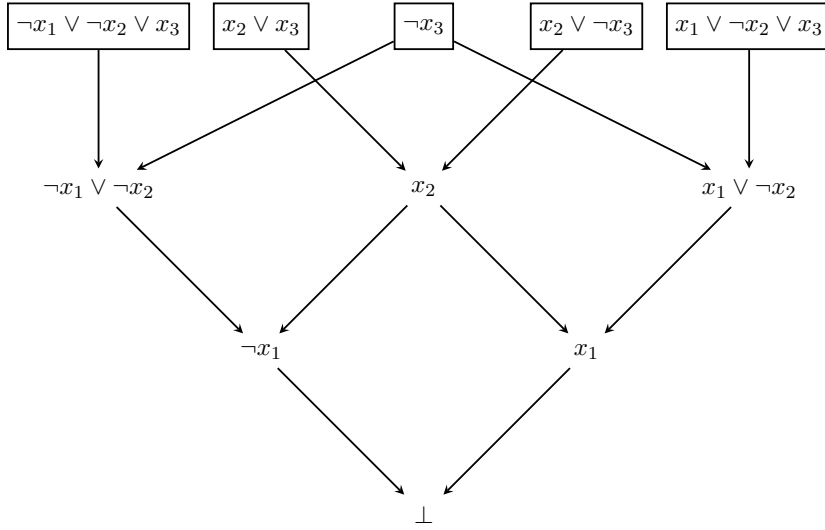


Figure 3. DAG-like representation of the refutation in ??.

We observe that the definition of Resolution refutation allows rules to be applied more than once onto each clause of the proof. There may also be multiple copies of a single clause, each obtained by applying different rules. When each copy of a clause is used only once as premise in the whole proof, we say that the refutation is *tree-like* (named after the fact that the graph becomes a tree). This restriction implies that if we want to use a clause multiple times then it must be derived again, usually starting from the same axioms.

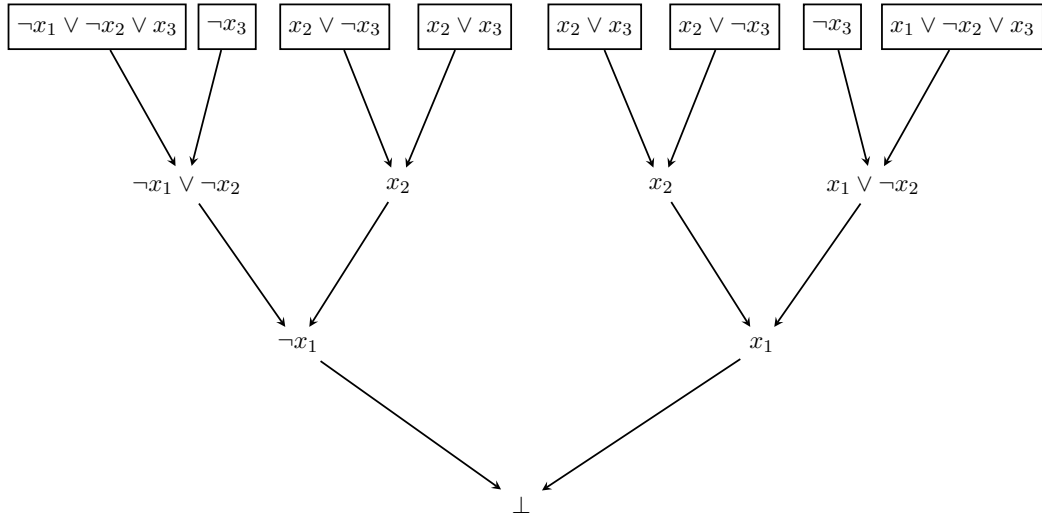


Figure 4. Tree-like adaptation of the refutation in ??.

This sub-system of Resolution is called *tree-like Resolution*, denoted with Res^* . The same idea can also be applied to other dynamic systems, obtaining sub-systems like $\text{Res}^*(k)$, F^* and PC^* . Generally, tree-like sub-system require longer proofs due to the impossibility of reusing derived clauses.

3.3 Simulations between proof systems

To compare the relative strength of different propositional proof systems, we introduce the notion of efficient simulation. This notion comes in a natural way by adapting the idea of polynomial time reduction.

Definition 3.2. Given two proof systems P, Q , we say that Q polynomially simulates P , written $P \leq_p Q$ if there is a polynomial time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for all tautologies φ if π is a P -proof of φ then $f(\pi)$ is a Q -proof of φ .

Intuitively, this means that there is an efficient algorithm transforming P -proofs into Q -proofs for the same formula. Consider the binary relation $\leq_p$. Similarly to reductions, this relation forms a partial order. In particular, for any pair P, Q of proof systems we say that:

- P and Q are *polynomially equivalent* when $P \leq_p Q$ and $Q \leq_p P$. We often write $P \equiv_p Q$ when this holds.
- P is *superpolynomially weaker* than Q when $P \leq_p Q$ but $Q \not\leq_p P$
- P and Q are *incomparable* when $P \not\leq_p Q$ and $Q \not\leq_p P$

It's easy to see that for any dynamic proof system Q it holds that $Q^* \leq_p Q$ since any Q^* -proof is also a Q -proof. Vice versa, however, it commonly holds that $Q \not\leq_p Q^*$, making tree-like systems typically weaker than their DAG-like counterpart. For instance, standard techniques can prove that the Ordering Principle formula OP_n is an hard formula for tree-like Resolution but an easy one for Resolution, implying that $\text{Res} \not\leq_p \text{Res}^*$.

When it comes to comparing Resolution with Nullstellensatz and Polynomial Calculus, they're both incomparable with it. This is due to the fact that negative literals easily blow up the size of the proof due to the increase in the number of monomials when expanded. To fix this, we can consider a variant of Polynomial Calculus referred to as *Polynomial Calculus with Resolution* (PCR) – a name that pretty much says it all – where for each variable x_i we add a new variable $\bar{x}_i$ in order to map negative literals $\neg x_i$ directly to this new variable.

$$\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \mapsto \prod_{i \in I} x_i \prod_{j \in J} \bar{x}_j = 0$$

All the other standard simulations and separations between proof systems are omitted and summarized in ??.

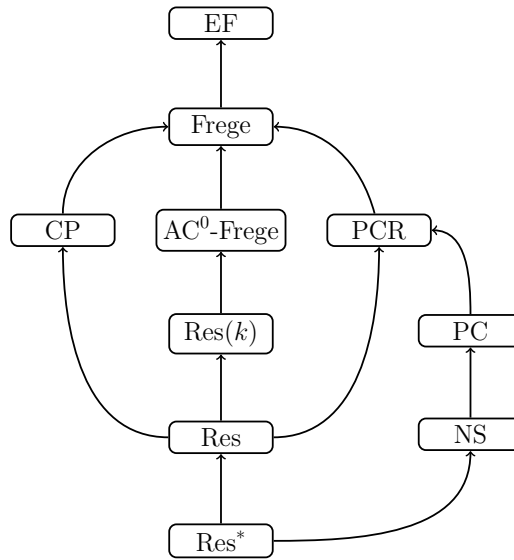


Figure 5. Simulations between standard proof systems.

add references for
simulations (also
inside the image)

3.4 Complexity measures for proof systems

Complexity measures lie at the heart of proof complexity. These measures provide a rigorous language to measure if a minimal proof must be short or long, allowing us to evaluate how different systems perform on proving a common statement. Different measures highlight different facets of a proof, often interlinking them. By analyzing these measures, we gain a deeper understanding of the inherent difficulty of logical reasoning, even from an algorithmic perspective.

Resolution is the quintessential proof system when it comes to complexity measures. Consider a Res-proof π . For any clause C , let $|C|$ denote the number of literals in C . Our main complexity measure of interest is The *size* of π , written as $|\pi|$, corresponding to the total number of literals inside it, i.e. $|\pi| = \sum_{D_i \in \pi} |D_i|$. For instance, the refutation in ?? has size 19. In a natural way, the size of the proof corresponds to the number of symbols used to describe the witness of the verifier-based definition of the proof system. For any formula φ with m clauses over n variables, the Resolution size of φ , written as $\text{size}_{\text{Res}}(\varphi)$, is the minimum size across all Res-proofs of φ (equivalently, across all refutations of $\neg\varphi$).

$$\text{size}_{\text{Res}}(\varphi) = \min_{\pi: \text{Res} \vdash \neg\varphi} |\pi|$$

Finding the Resolution size of a formula tells us the exact minimum number of symbols in a witnessing string for the given formula. Proving the existence of at least one formula φ , with m clauses over n variables, whose Resolution size is super-polynomial with respect to $|\varphi| = \Theta(n + m)$ is sufficient to conclude Resolution is not polynomially bounded. Haken was the first to show that one such formula is the Pigeonhole Principle formula PHP_n^{n+1} , which has size $2^{\Theta(n)}$. This result not only implies that Resolution is polynomially bounded, but it also implies that every CNF

add ref

formula that implicitly describes the Pigeonhole Principle is hard for Resolution. Since this combinatorial principle is at the foundation of mathematics, this makes Resolution weak against a wide subset of formulas.

A natural complexity measure related with size is the *length* of a proof, which measures the total number of clauses inside π , i.e. the number of steps in the proof. The Resolution length of a formula φ , written as $\text{length}_{\text{Res}}(\varphi)$, is the minimum length across all Res-proofs of φ . One can easily prove that $\text{length}_{\text{Res}}(\varphi) \leq \text{size}_{\text{Res}}(\varphi) \leq n \cdot \text{length}_{\text{Res}}(\varphi)$ holds for any φ . For this reason, the concepts of size and length are often treated as synonyms in the case of Resolution.

Another complexity measure inspired by Haken's proof is the *width* of a proof π , defined as the maximum number of literals across all clauses of π , i.e. $\max_{D_i \in \pi} |D_i|$. The Resolution width of φ , written as $\text{width}_{\text{Res}}(\varphi)$, is defined as the minimum across all proofs. Ben-Sasson and Wigderson proved that Resolution size and width are related one another through the trade-off.

add ref

Theorem 3.3. $\text{size}_{\text{Res}}(\varphi) = \exp \left(\Omega \left(\frac{(\text{width}_{\text{Res}}(\varphi) - \text{width}(F))^2}{n} \right) \right)$

We observe that in the exponent of the right hand side of the equation we have the difference between the Resolution width and the width of the axioms in φ . Because of this, for the case of families of formulas with large initial clauses, like the Pigeonhole Principle formula PHP_n^{n+1} , this method can prove only trivial size lower bounds (in fact, Haken's proof uses width of *monotone* clauses).

All Resolution measures carry over to Res^* , $\text{Res}(k)$ and CP, with some small variations: in $\text{Res}(k)$ the concept of width translates to the number of k -DNFs inside the conjunctions, while in CP there is no real concept of width. These adaptations imply that the above trade-off theorem doesn't have an analogous in these two worlds.

Moving to other proof systems, size and length measures are inherited by Frege systems in a natural way, but the relation between $\text{size}_{\text{Frege}}(\varphi)$ and $\text{length}_{\text{Frege}}(\varphi)$ and length is not so straightforward anymore since the formulas inside the proof are not restricted to being clauses. Nonetheless, Reckhow proved that they are still polynomially related one another.

add ref

Theorem 3.4. $\text{size}_{\text{Frege}}(\varphi) = O(\text{length}_{\text{Frege}}(\varphi) \cdot (|\varphi| + \text{length}_{\text{Frege}}(\varphi))^2)$

Other than this, no general trade-offs are known for Frege systems. In addition, there is no concept of width (again, formulas are not restricted to being clauses). Nonetheless, there is another complexity measures of our interest called *nested depth*, that being the maximum nesting level of logical connectives across all formulas of the proof. For general Frege system, there are no known formulas that require exponential size proofs. However, when restricted to $\Theta(1)$ nested depth, some formulas are known. This sub-system of Frege is known as AC^0 -Frege. Furthermore, it can be easily proven that Frege is equivalent to Resolution when restricted to depth-1 formulas.

Both in Nullstellensatz and Polynomial Calculus, size is defined differently. The size of a polynomial $||p||$ is defined as the number of non-zero monomials

when expanded out as a linear combination of monomials. The size of a proof $||\pi||_Q$, with $Q \in \{\text{NS}, \text{PC}\}$, is the sum of the sizes of all the polynomials inside it. Regarding polynomials, we have another complexity measure: *degree*. The degree of a polynomial $\deg(p)$ is the maximum degree of all monomials inside it. The degree of a proof is $\deg_Q(\pi)$ is the maximum degree across all polynomials inside it. The values of $\text{size}_Q(\varphi)$, $\deg_Q(\varphi)$ and are defined accordingly.

In particular, for a NS-proof $\pi = \{g_1, \dots, g_m, h_1, \dots, h_n\}$ over the system $\{p_1, \dots, p_m\} \cup \{x_i^2 - x_i\}_{i \in [n]}$ the size and degree of π are defined by:

$$||\pi||_{\text{NS}} = \sum_{j=1}^m ||g_j|| ||p_j|| + \sum_{i=1}^n 2 ||h_i||$$

$$\deg_{\text{NS}}(\pi) = \max_{j \in [m], i \in [n]} (\deg(g_j) + \deg(p_j), 2 + \deg(h_i))$$

For a PC-proof $\pi' = (f_1, \dots, f_t)$, instead, the size of π' is defined as $||\pi'||_{\text{PC}} = \sum_{i=1}^t ||f_i||$ and the degree is $\max_{i \in [t]} (\deg(f_i))$.

3.5 Game characterizations of Resolution measures

3.5.1 The Prover-Delayer Game (TODO)

3.5.2 The Spoiler-Duplicator Game (TODO)

Chapter 4

Automatability

4.1 The quest for Automated Theorem Proving (TODO)

4.2 Quasi-automatability of Tree-like Resolution (TODO)

For the case of Tree-like Resolution, there is a non-trivial quasi-polynomial time automating algorithm. The algorithm is based on two simple property of tree-like refutations in Resolution, formalized by the next two lemmas.

Lemma 4.1. *Let φ be any CNF formula. If φ has a tree-like Resolution refutation of size s then $\varphi \cup \{\ell\}$ has a tree-like Resolution refutation of size at most $s/2$ for some literal ℓ .*

Proof. Let $\pi : \text{Res} \vdash \neg\varphi$. In π , the empty clause $\perp$ is obtained through a resolution step between two clauses equal to x_i and $\neg x_i$, respectively. Let π_{x_i} and $\pi_{\neg x_i}$ be the two subtrees of π that have x_i and $\neg x_i$ as roots. Notice that $|\pi| = |\pi_{x_i}| + |\pi_{\neg x_i}| + 1$. W.l.o.g, assume that π_{x_i} is the smallest of the two subtrees. Then, it holds that $|\pi_{x_i}| < s/2$ since π is a binary tree. Thus, the refutation π'_{x_i} obtained by resolving the root of π_{x_i} with $\neg x_i$ is a refutation for $\varphi \cup \{\neg x\}$ of size at most $s/2$. $\square$

Lemma 4.2. *Let φ be any CNF formula. If $\varphi \cup \{\ell\}$ for some literal ℓ has a tree-like Resolution refutation of size s then $\neg\ell$ can be derived in tree-like Resolution starting from φ with at most s steps.*

Proof. Let π be a tree-like refutation of $\varphi \cup \{\ell\}$ of size s . We construct a new derivation π' from π in the following way: for each clause C in π derived by resolving $C \vee \ell$ and ℓ , we substitute C with $C \vee \ell$ (in other words, we skip the resolution step). Repeating this process down the whole tree of π transforms the root clause $\perp$ into ℓ . $\square$

The idea behind the algorithm is to use these two lemmas to apply a smart brute force approach where wrong paths are discarded. For any formula φ , let $s = \text{size}_{\text{Res}^*}(\varphi)/2$. We start by searching for a literal ℓ such that there is a tree-like

Resolution refutation for $\varphi|_{\ell=0}$ of size $s/2$. Once one is found, we recursively search for a proof of $\varphi|_{\ell=1}$ of size s .

Theorem 15.2 There is an algorithm that on input an unsatisfiable CNF formula with n variables, finds a Tree-like Resolution refutation for it in time $(\log n)^{O(1)}$, where n is the minimal size of a Tree-like Resolution refutation of φ .

4.3 Non-automatability of Resolution

4.3.1 The Ref formula

Given a CNF formula φ and a parameter $s \in \mathbb{N}$, we define a formula $\text{Ref}_s(\varphi)$ representing the statement “ φ has a Resolution refutation consisting of s clauses”. The simplicity of Resolution allows us to encode this meta-complexity statement through a natural encoding. The idea is to formalize $\text{Ref}_s(\varphi)$ as a blueprint for a generic Resolution refutation for φ with at most s clauses. Each clause is represented by a *block*, a set of variables that describe the literals that appear in the clause, how it was obtained (resolved or weakened from another clause) and two (or one) pointer variables that keep track of the parent blocks it was derived from. The axioms of the formula are formal descriptions of coherence rules between these variables. For instance, if the variable saying that a block B was derived through a resolution step is set to true then all the other variables that talk about the derivation itself (parent pointers, resolving literal, ...) must also be set.

The initial standard encoding of this idea was the one used in Pudlák [Pud03], referred to in the literature as the *unary encoding*. This encoding uses unary pointers, meaning that for every block $B \in [s]$ there are up to $2(s-1)$ variables that point at the blocks from which B may be derived. Pudlák proved the easiness of $\text{Ref}_s(\varphi)$ when φ is satisfiable through this encoding, but the proof can be adapted for other encodings.

In order to prove their result, Atserias and Müller [AM20] adapted this encoding by introducing *enable variables*. When the enable variable is true, the block is considered as part of the underlying refutation described by the blueprint (as if we’re forcing the refutation to use this block). When this isn’t the case, instead, the block gets completely ignored by the refutation, automatically satisfying all the axioms that mention it. This encoding is known as the *relativized unary encoding*. This encoding was first introduced to prove the hardness of $\text{Ref}_s(\varphi)$ when φ is unsatisfiable due to failure of achieving such result with the non-relativized encoding. Nonetheless, Garlik [G19] proved that the hardness of this formula doesn’t come from the introduction of this variables by proving that the size lower bound also holds for the non-relativized encoding.

Finally, the last common encoding is the *relativized binary encoding*, first defined in [dRGN+21]. This encoding describes the parent pointers of each block in binary, meaning that for each block $B \in [s]$ there are only $O(\log s)$ variables describing its pointers. This encoding drastically reduces the width of the formula, making it possible to prove the size lower bound simply by using a size-width tradeoff.

In this work we'll use the relativized unary encoding as it is both the most natural one for the problem and the easiest to work with (see Arteche [A25] for a more in depth comparison behind the encodings). Given n variables $x_1, \dots, x_n$, let $\text{Lit}_n = x_1, \dots, x_n, \neg x_1, \dots, \neg x_n$. We define the variables of the formula $\text{Ref}_s(\phi)$, parametrized over a generic formula ϕ with n variables and m clauses, as follows:

- a-lit $_{\ell}^A$: literal $\ell \in \text{Lit}_n$ is present in the clause $A \in [m]$ of ϕ ;
- enable B : block $B \in [s]$ is enabled;
- derived B : block $B \in [s]$ was obtained by a Resolution step;
- weak $_A^B$: block $B \in [s]$ is obtained by weakening from clause $A \in [m]$ of ϕ ;
- lit $_{\ell}^B$: literal $\ell \in \text{Lit}_n$ is present in the block $B \in [s]$;
- res $_{x_i}^B$: block $B \in [s]$ was obtained by resolving over the variable x_i ;
- lpoint $_{B'}^B$: block $B \in [s]$ was resolved on the left from block $B' \in [s], B' < B$;
- rpoint $_{B'}^B$: block $B \in [s]$ was resolved on the right from block $B' \in [s], B' < B$.

When working with a specific formula φ , we restrict $\text{Ref}_s(\phi)$ to the formula $\text{Ref}_s(\varphi) := \text{Ref}_s(\phi)|_{\alpha_{\varphi}}$ through a partial assignment α_{φ} that sets the values of all the variables a-lit $_{\ell}^A$ in a way that describes the clauses of φ , i.e. $\alpha_{\varphi}(\text{a-lit}_{\ell}^A)$ is set to 1 if ℓ is part of the A -th clause of φ , otherwise we set it to 0. Moreover, we also observe that the weakening variables weak $_A^B$ are defined only for axiom clauses. Of course, this can be extended to non-axiom clauses but, for the purpose of making proofs easier and shorter, we'll assume that the weakening step can be used only on axioms. The clauses of $\text{Ref}_s(\varphi)$ are then defined as follows:

$$\begin{aligned}
& \left(\text{enable}^B \wedge \text{res}_{x_i}^B \wedge \text{lpoint}_{B'}^B \wedge \text{lit}_{\ell}^{B'} \right) \rightarrow \text{lit}_{\ell}^B && \text{for } B, B' \in [s], B' < B, i \in [n], \ell \in \text{Lit}_n \setminus \{x_i\}, && (\text{REF-1}) \\
& \left(\text{enable}^B \wedge \text{res}_{x_i}^B \wedge \text{rpoint}_{B'}^B \wedge \text{lit}_{\ell}^{B'} \right) \rightarrow \text{lit}_{\ell}^B && \text{for } B, B' \in [s], B' < B, i \in [n], \ell \in \text{Lit}_n \setminus \{\neg x_i\}, && (\text{REF-2}) \\
& \left(\text{enable}^B \wedge \text{weak}_A^B \wedge \text{a-lit}_{\ell}^A \right) \rightarrow \text{lit}_{\ell}^B && \text{for } B \in [s], A \in [m], \ell \in \text{Lit}_n, && (\text{REF-3}) \\
& \left(\text{enable}^B \wedge \text{derived}^B \right) \rightarrow \bigvee_{i \in [n]} \text{res}_{x_i}^B && \text{for } B \in [s], && (\text{REF-4}) \\
& \left(\text{enable}^B \wedge \text{derived}^B \right) \rightarrow \bigvee_{\substack{B' \in [s] \\ B' < B}} \text{lpoint}_{B'}^B && \text{for } B \in [s], && (\text{REF-5}) \\
& \left(\text{enable}^B \wedge \text{derived}^B \right) \rightarrow \bigvee_{\substack{B' \in [s] \\ B' < B}} \text{rpoint}_{B'}^B && \text{for } B \in [s], && (\text{REF-6}) \\
& \left(\text{enable}^B \wedge \neg \text{derived}^B \right) \rightarrow \bigvee_{A \in [m]} \text{weak}_A^B && \text{for } B \in [s], && (\text{REF-7}) \\
& \left(\text{enable}^B \wedge \text{lpoint}_{B'}^B \right) \rightarrow \text{enable}^{B'} && \text{for } B, B' \in [s], B' < B && (\text{REF-8}) \\
& \left(\text{enable}^B \wedge \text{rpoint}_{B'}^B \right) \rightarrow \text{enable}^{B'} && \text{for } B, B' \in [s], B' < B && (\text{REF-9}) \\
& \neg \text{lit}_{\ell}^s && \text{for } \ell \in \text{Lit}_n, && (\text{REF-10}) \\
& \text{enable}^s. && && (\text{REF-11})
\end{aligned}$$

The above clauses have been written as implications for the sake of readability. This encoding of $\text{Ref}_s(\phi)$ is defined over $N = \Theta(s^2 + sm + sn + nm)$ variables and $M = \Theta(s^2n^2 + smn)$ clauses, meaning that $|\text{Ref}_s(\phi)| = \text{poly}(s, n, m)$. The values of N and M will be crucial for our proofs.

Proposition 4.1. *φ has a Resolution refutation of length at most s if and only if $\text{Ref}_s(\varphi)$ is satisfiable.*

Proof. Let π be a refutation of φ of length $s' \leq s$. We can easily define a satisfying assignment for $\text{Ref}_s(\varphi)$ by enabling exactly s' blocks to then set all their variables by following the description of π .

Vice versa, let α be a satisfying assignment for $\text{Ref}_s(\varphi)$. Since the clauses of $\text{Ref}_s(\varphi)$ acts as a blueprint for a resolution refutation, we can read the contents of α starting from the root block s down to the axioms to obtain the steps of a refutation π' of φ . Since $\text{Ref}_s(\varphi)$ has at most s enabled blocks, we get a refutation of length at most s for φ . $\square$

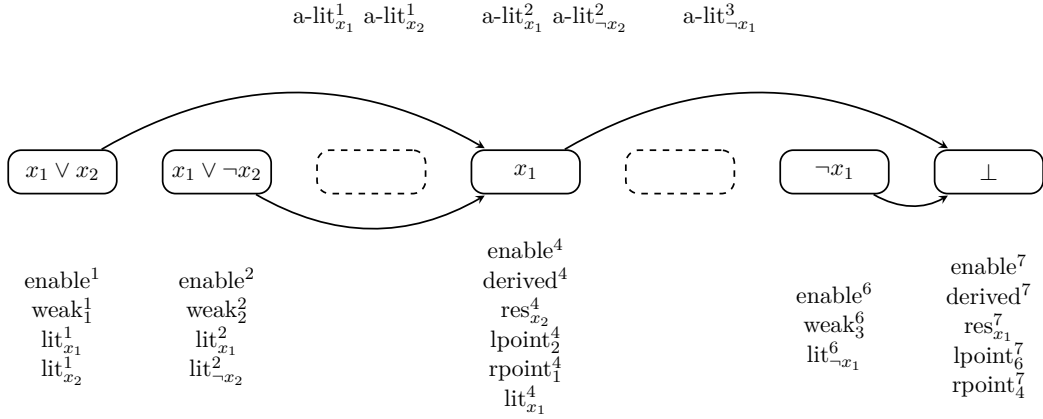


Figure 6. Mapping between a refutation of $\varphi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge \neg x_1$ and a satisfying assignment for $\text{Ref}_7(\varphi)$. Displayed variables are set to 1, the remaining ones are set to 0.

The above proposition not only formalizes the soundness of the formula $\text{Ref}_s(\varphi)$, but it also shows that, when it is a tautology, the minimum size of a Q -proof π of $\neg \text{Ref}_s(\varphi)$ determines the ability of a proof system to reason efficiently or not about Resolution lower bounds. In particular, when $\text{Ref}_s(\varphi)$ is unsatisfiable one of two cases may hold: either φ is unsatisfiable but it requires refutations of length at least $s + 1$ or φ is satisfiable, thus it has no refutations at all. This second case may also be seen as if φ requires “refutations of infinite length”.

A fundamental concept that will be used in our proofs is *block-width*. Consider any formula φ whose variables $x_1, \dots, x_n$ can be partitioned into distinct blocks, similar to what is done in $\text{Ref}_s(\phi)$. When a variable x is part of a block B , we say that x *mentions* B . The Resolution block-width of a clause C over the variables of φ , written bw_φ , is defined as the number of different blocks mentioned by the variables of the literals in C , without counting the mentions of the root block s . The block-width $\text{bw}_{\text{Res}}(\pi)$ of a Resolution refutation π for $\text{Ref}_s(\varphi)$ the maximum block-width over all the clauses in π . By extension, the block-width $\text{bw}(\varphi)$ of a formula φ is the minimum block-width across all refutations.

Our first goal is to prove that refutation formulas can act as a discriminator between satisfiability and unsatisfiability. This will later be used to show that if Resolution is automatable then SAT can be solved in polynomial time, implying that $\text{P} = \text{NP}$.

Theorem 4.2. *Let φ be a formula with n variables and m clauses. Then:*

- (1) *If φ is satisfiable then $\text{Ref}_s(\varphi)$ has a Resolution refutation of size $N^{O(1)}$ and block-width $\Theta(1)$.*
- (2) *If φ is unsatisfiable then $\text{Ref}_s(\varphi)$ requires Resolution refutation of block-width $N^{\Omega(1)}$.*

We split the proof of this theorem in two lemmas (?? and ??), proven respectively in the next two sub-sections.

4.3.2 Upper bound on Ref for satisfiable formulas

We prove that when φ is satisfiable the formula $\text{Ref}_s(\varphi)$ admits a short Resolution refutation, corresponding to part (1) of ??. This result was first proven by Pudlák [Pud03] for the unary encoding of $\text{Ref}_s(\varphi)$ to show that the canonical NP-pair of Resolution is *symmetrical*, i.e. that the pair $(\text{Sat}, \text{Ref}^{\text{Res}})$ is polynomial time reducible to the pair $(\text{Ref}^{\text{Res}}, \text{Sat})$. Atserias and Müller showed that such upper bound is preserved in the relativized unary encoding, while de Rezende [dR20] showed it for the binary encoding. Here, we use an adaptation of the proof for the relativized unary encoding shown in [Arteche25].

Lemma 4.3. *Let φ be a satisfiable CNF formula with n variables and m clauses. Then, for all $s \in \mathbb{N}$ the formula $\text{Ref}_s(\varphi)$ has a Resolution refutation of length $O(s(m + sn^2))$.*

Proof. Let α be a satisfying assignment for φ . We'll construct a refutation for $\text{Ref}_s(\varphi)$ using α as a guide. For all $B \in [s]$, we'll derive (in increasing order of B) the following clause $\text{True}(B, \alpha)$ asserting that “if block B is enabled then α satisfies it”:

$$\text{True}(B, \alpha) := \neg \text{enable}^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n: \\ \alpha(\ell)=1}} \text{lit}_\ell^B$$

After deriving $\text{True}(s, \alpha)$, the proof concludes by resolving it sequentially against all clauses REF-10 and REF-11 defined for s .

We start by deriving the clause $Z_1(B) := \text{derived}^B \vee \text{True}(B, \alpha)$ for each $B \in [s]$. Since $\alpha(\varphi) = 1$, for each clause $C_j \in \varphi$ there is at least one literal $\ell_j^\alpha \in C_j$ such that $\alpha(\ell) = 1$. Now, consider the following auxiliary clause $W(B, j)$:

$$W(B, j) := \neg \text{enable}^B \vee \neg \text{weak}_i^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n: \\ \alpha(\ell)=1}} \text{lit}_\ell^B$$

Notice that $W(B, j)$ is a weakening of the axiom clause REF-3(B, j, ℓ_j^α) since the variable $\text{a-lit}_{\ell_j^\alpha}^j$ gets deleted by the restriction once φ has been fixed inside $\text{Ref}_s(\phi)$. This accounts for a total of m weakening steps.

By resolving axiom REF-7(B) sequentially, for all $j \in [m]$, with each $W(B, j)$ over the variable weak_j^B , obtaining the clause $Z_1(B)$. This accounts for a total of

$m - 1$ resolution steps. Since this procedure has to be applied for each block $B \in [s]$, we get a total of $O(sm)$ steps to obtain $Z_1(1), \dots, Z_1(s)$.

Next, we'll derive $\text{True}(B, \alpha)$ for each $B \in [s]$ through an inductive procedure. When $B = 1$, the axiom $\text{REF-5}(1)$ degenerates to $\neg \text{enable}^1 \vee \neg \text{derived}^1$ since no block comes before it. Therefore, by resolving $\text{REF-5}(1)$ with $Z_1(1)$ over the variable derived^1 we obtain $\text{True}(1, \alpha)$.

Fix $B \in [s]$ with $B > 1$ and assume that for each $B' \in [B - 1]$ we have already derived $\text{True}(B', \alpha)$. Fix a block $B' \in [B - 1]$ and an index $i \in [n]$. Suppose that $\alpha(x_i) = 0$ (we omit the dual case since it can be constructed similarly by following the right-hand pointers). For each $k \in [n]$, set $\ell_k = x_k$ if $\alpha(x_k) = 1$ and $\ell_k = \neg x_k$ if $\alpha(x_k) = 0$.

By resolving $\text{True}(B', \alpha)$ sequentially, for all $k \in [n]$, with each axiom $\text{REF-1}(B, B', i, \ell_k)$ over the variable $\text{lit}_{\ell_k}^{B'}$ we get the auxiliary clause $Z(B, B', i)$ defined as:

$$U_1(B, B', i) := \neg \text{enable}^B \vee \neg \text{enable}^{B'} \vee \neg \text{res}_{x_i}^B \vee \neg \text{lpoint}_{B'}^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n: \\ \alpha(\ell)=1}} \text{lit}_{\ell}^B$$

This requires $O(n)$ steps. By resolving each $U_1(B, B', i)$ with $\text{REF-8}(B, B')$ over the variable $\text{enable}^{B'}$, we obtain the clauses $U_2(B, B', i)$ of the form

$$U_2(B, B', i) := \neg \text{enable}^B \vee \neg \text{res}_{x_i}^B \vee \neg \text{lpoint}_{B'}^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n: \\ \alpha(\ell)=1}} \text{lit}_{\ell}^B$$

This requires another $O(n)$ steps. By repeating above process across all $i \in [n]$, we derive the clauses $\{U_2(B, B', i)\}_{i \in [n]}$ for a fixed B' with $O(n^2)$ steps. Now, we cut $\text{REF-4}(B)$ sequentially, for all $i \in [n]$, with $U_2(B, B', i)$ over the variable $\text{res}_{x_i}^B$ to obtain the clauses $U_3(B, B')$ of the form

$$U_3(B, B') := \neg \text{enable}^B \vee \neg \text{derived}^B \vee \neg \text{lpoint}_{B'}^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n: \\ \alpha(\ell)=1}} \text{lit}_{\ell}^B$$

Repeating this over all $B' \in [B - 1]$, we obtain all the clauses $\{U_3(B, B')\}_{B' \in [B-1]}$ with $O(sn^2)$ steps. Finally, by cutting $\text{REF-5}(B)$ sequentially, for all $B' \in [B - 1]$, with $U_3(B, B')$ over the variable $\text{lpoint}_{B'}^B$ we get the clause $\neg \text{derived}^B \vee \text{True}(B, \alpha)$, which can then be resolved with the corresponding clause $\text{derived}^B \vee \text{True}(B, \alpha)$ to get $\text{True}(B, \alpha)$. By applying this whole inductive procedure across all block $B \in [s]$, we get a total of $O(sm + s^2n^2)$ steps. $\square$

4.3.3 Block-width lower bound for Ref

We prove part (2) of ??, first proven by Atserias and Müller [AM20]. Their original proof is based on a very complex restriction argument. Here, we shall show the easier proof given by de Rezende [dR21]. This simplified approach is based on *decision tree reductions*. Informally, a decision tree reduction between formulas φ and ψ is a

reduction relating the variables of ψ to the variables of φ via shallow decision trees, and moreover, showing that the axioms of φ imply those of ψ .

Definition 4.1. Let φ and ψ be, respectively, CNF formulas over the variables $x_1, \dots, x_n$ and $y_1, \dots, y_s$. A depth- d decision tree reduction between φ and ψ , written as $\varphi \stackrel{dt}{\leq}_d \psi$ is a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^t$

- Each output bit f_i is computed by an independent depth- d decision tree $\mathcal{T}_i : \{0, 1\}^n \rightarrow \{0, 1\}$
- For each axiom C of ψ , viewed as a function $C : \{0, 1\}^t \rightarrow \{0, 1\}$, the composition $C \circ f$ can be computed by a depth- $(d \cdot |C|)$ decision tree $\mathcal{T}_C$
- For each axiom C of ψ , every clause of $\mathcal{T}_C$ is a weakening of an axiom of φ

The key idea behind these reduction is to use the CNF representation property of decision trees to get a formula that is equal to ψ but where each variable y_i is replaced by the corresponding decision tree. This idea naturally translates to a relationship between the widths of the two formulas. To formally prove this, we can use the Prover-Delayer game defined in .

add ref

Proposition 4.3. If $\varphi \stackrel{dt}{\leq}_d \psi$ then $\text{width}_{\text{Res}}(\varphi) \leq d \cdot \text{width}_{\text{Res}}(\psi)$.

Proof. Let f be the function witnessing $\varphi \stackrel{dt}{\leq}_d \psi$. Given a width- w Prover strategy $\mathcal{G}$ for ψ , we define a width- dw Prover strategy $\mathcal{F}$ for φ by simulating $\mathcal{G}$ round-by-round through f .

We'll use the following invariant during each round of the game: if the game state for $\mathcal{G}$, i.e. the partial assignment to y , records a value $y_i = b$ for some $b \in \{0, 1\}$, then the current game state ρ for $\mathcal{F}$, i.e. the partial assignment to x , satisfies $f_i(\rho) = b$ while having at most d values of x_j recorded in ρ .

On each round, if $\mathcal{G}$ queries a variable y_i then $\mathcal{F}$ computes $f_i(x) = b$ and simulates $\mathcal{G}$ by responding $y_i = b$ – in other words, $\mathcal{F}$ acts as the Adversary for $\mathcal{G}$. When $\mathcal{G}$ forgets a variable y_i , instead, $\mathcal{F}$ forgets all the variables x_j that aren't required to know the values of $f_j(x)$ for all those $j \in [t]$ such that the value of y_j is not forgotten by $\mathcal{G}$.

This simulation ensures that the strategy of $\mathcal{F}$ is at most dw . Moreover, when the game ends for $\mathcal{G}$ it will also end for $\mathcal{F}$: if $\mathcal{G}$ 't game state falsifies an axiom of G , it will also falsify an axiom of $\mathcal{F}$ through the definition of f . $\square$

The idea in the above proof can be easily extended to block-width. To do so, we consider the so-called *block-aware reductions*. The idea is to parallelize the decision trees that compute the reduction for the variables of a block into a single decision tree for the whole block.

Definition 4.2. Let f be a depth- d decision tree reduction witnessing $\varphi \stackrel{dt}{\leq}_d \psi$. Suppose that the variables $y_1, \dots, y_s$ of ψ are partitioned into blocks. We say that the reduction f is block-aware if for each block $B \leq [t]$ there is a depth- d decision tree that computes all the values $f_B(x) := (f_i(x) : \{0, 1\}^n \rightarrow \{0, 1\}^{|B|})_{i \in B}$ simultaneously.

Prover-Adversary games can equally well characterize block-width, defined for a game state as the number of blocks that the state records values from.

Proposition 4.4. *If $\varphi \leq_d^{dt} \psi$ via a block-aware reduction then $\text{width}_{\text{Res}}(\varphi) \leq d \cdot \text{bw}_{\text{Res}}(\psi)$.*

Proof. The proof is similar to that of ??, assuming an additional invariant: for each block B such that $\mathcal{G}$ records the value of some y_i with $i \in B$, the simulation $\mathcal{F}$ knows $f_B(x)$ by recording at most d values of the x_j variables. From this new constraint it follows that if $\mathcal{G}$ has block-width w then $\mathcal{F}$ has width at most dw . $\square$

The above proposition is the main tool behind the proof of our desired block-width lower bound for $\text{Ref}_s(\varphi)$. Now, all we require is a final ingredient: it suffices to find a proper initial propositional formula from which we can obtain a nice width lower bound and a nice block-aware reduction. It turns out that this ingredient is the *Retraction Weak Pigeonhole Principle* rPHP_m^{2m} , a variant of the standard Pigeonhole Principle formula.

This variant uses $2m$ pigeons and m holes, while also providing an efficient way to invert the mapping. This is achieved through two functions $f : [2m] \rightarrow [m]$ and $g : [m] \rightarrow [2m]$, where the addressing of the mappings is given in binary:

- *Pigeon mapping:* for every pigeon $i \in [2m]$ there are $\log m$ variables $f_{i,k}$, $k \in [\log m]$, that describe the binary encoding of the hole $f(i)$ that will be occupied by the i -th pigeon.
- *Hole mapping:* for every hole $j \in [m]$ there are $\log 2m$ variables $g_{j,h}$, $h \in [\log 2m]$, that describe the binary encoding of the pigeon $g(j)$ that will occupy the j -th hole.

The axioms of rPHP_m^{2m} state for every $i \in [2m]$, $j \in [m]$ if $f(i) = j$ then $g(j) = i$ – equivalently, g is a left-inverse of f . To encode this idea, for each pair $(i, j) \in [2m] \times [m]$ we define the formula $\neg[f(i) = j]$ as:

$$\neg[f(i) = j] := \bigvee_{k \in [\log m]} \ell(i, j, k)$$

where $\ell(i, j, k) = f_{i,j_k}$ if $j_k = 1$, otherwise $\ell(i, j, k) = \neg f_{i,j_k}$ – recall that j_k is the k -bit of j 's binary encoding. Then, the condition $f(i) = j \rightarrow g(j) = i$ is described by:

$$\bigwedge_{h \in [\log m]} \neg[f(i) = j] \vee \gamma(j, i, h)$$

where $\gamma(j, i, h) = g_{j,i_h}$ if $i_h = 1$, otherwise $\gamma(j, i, h) = \neg g_{j,i_h}$. Ranging across all pairs, the formula rPHP_m^{2m} has $O(m^2 \log m)$ clauses, each containing $O(\log m)$ literals. We observe that such axioms imply all the standard pigeonhole axioms, hence no other axioms are required.

We start by proving the width lower bound on the rPHP_m^{2m} formula using the Spoiler-Duplicator game defined in .

Lemma 4.4. $\text{width}_{\text{Res}}(\text{rPHP}_m^{2m}) = \Omega(m)$.

Proof. We use a Duplicator strategy similar to the classical one used for the standard pigeonhole principle. Set $w = \lfloor m/2 \rfloor$. The Duplicator maintains a family $\mathcal{H}$ of partial assignments α with $|\text{dom}(\alpha)| \leq w$. Each $\alpha \in \mathcal{H}$ is associated with a partial injective mapping $q_\alpha : P_\alpha \rightarrow H_\alpha$, where $P_\alpha \subseteq [2m]$ and $H_\alpha \subseteq [m]$ with $|P_\alpha| = |H_\alpha| \leq |\text{dom}(\alpha)| \leq w$, such that α is consistent with q_α .

Let α be the Duplicator's assignment of the current game state, where $|\text{dom}(\alpha)| < w$. Suppose that the Spoiler queries a variable $f_{i,k}$ (analogous argument for variables $g_{j,h}$, where pigeons and holes are inverted). The idea is to associate each of the $\log m$ hole bits to a different pigeon. In particular, if $i \notin P_\alpha$ then $|H_\alpha| \leq k < m$ guarantees that at least one hole $j' \in [m] \setminus H_\alpha$ is unassigned. Hence, the Duplicator answers with j'_k to then set $\alpha' = \alpha \cup \{f_{i,k} \mapsto j'_k\}$ and $q_{\alpha'} = q_\alpha \cup \{i \mapsto j'\}$ to keep track of progress.

When the Spoiler forgets a variable $f_{i,k}$ (or $g_{j,h}$), the Duplicator removes the mappings in α and q_α associated with pigeon i (or hole j).

This strategy guarantees that the Duplicator preserves local consistency without exhausting available pigeons or holes for at least w rounds. $\square$

All that is left now is to prove that, when φ is unsatisfiable, rPHP_m^{2m} can be reduced to $\text{Ref}_s(\varphi)$, for a suitable choice of s , through (relatively) small decision trees, while also being block-aware. We give a generalization of the argument shown in de Rezende [dR21] while also sharpening the exact values used in the proof.

Let T_n be the following full tree-like Resolution refutation of the formula φ , defined on the variables $x_1, \dots, x_n$. We assume that the weakening rule is applied only on the leaves and that at each layer $i \in [n]$ of the tree the resolution rule is always applied on the variable x_i . Therefore, T_n has $2n$ cleaves corresponding to all the possible width- n clauses, each being a weakening of some axiom of φ . For any assignment α to rPHP_m^{2m} , the reduction is going to produce an assignment β to $\text{Ref}_s(\varphi)$ that represents a purported refutation of φ isomorphic to a subtree $T_\alpha \subseteq T_n$ of the full tree T_n .

The set of nodes of T_α that have smaller degree in T_α than in T_n is referred to as the *boundary* of T_α . Clearly, T_α is not a valid refutation of φ since the boundary nodes have at least one missing child, but the non-boundary nodes will be indistinguishable from the corresponding neighborhoods of T_n . For this reason, the only axiom violations of $\text{Ref}_s(\varphi)$ will be a result of the boundary nodes.

Lemma 4.5. $\text{rPHP}_m^{2m} \leq_{O(n \log m)}^{\text{dt}} \text{Ref}_{m(n+1)-1}(\varphi)$ via a block-aware reduction when φ is an unsatisfiable CNF formula.

Proof. Let $s = m(n+1) - 1$. Given a total assignment α to rPHP_m^{2m} , consider the subtree $T_\alpha \subseteq T_n$ defined as above. We'll use α to identify each block B of $\text{Ref}_s(\varphi)$ with some node in T_n . Once the position of the node is known, the values of the variables in block B will be completely defined. To achieve this, we define the

following partial mapping $h : [2m] \rightarrow [m] \cup \{*\}$:

$$h(i) = \begin{cases} f(i) & \text{if } g(f(i)) = i \\ * & \text{otherwise} \end{cases}$$

Observe that even if α is a total assignment to rPHP_m^{2m} , the function h is still necessarily undefined for some pigeons. The below claim explains the reason why we're using the retraction pigeonhole principle instead of the standard one.

Claim 4.1. *h and its inverse can be computed by a decision tree of depth $O(\log m)$*

Proof of the claim. Fix $i \in [2m]$. By querying the variables $\{f_{i,k}\}_{k \in [\log m]}$, we get the full decision tree $\mathcal{T}_f$ computing $f(i)$. Then, on each leaf j of $\mathcal{T}_f$ we attach the full decision tree $\mathcal{T}_g$ computing $g(j)$ by querying the variables $\{g_{j,h}\}_{h \in [\log 2m]}$. Let $\mathcal{T}_h$ be the decision tree obtained after attaching each $\mathcal{T}_g$. Every leaf of $\mathcal{T}_h$ such that $g(j) = i$ outputs j , while all the others output $*$. This decision tree computes h with depth $\log m + \log 2m$. For the inverse of h , a similar approach can be used by inverting the roles of $\mathcal{T}_f$ and $\mathcal{T}_g$. $\square$

Consider now the graph G_α with $n + 1$ layers $\ell \in \{0, \dots, n\}$, each containing $\min\{2^\ell, m\}$ nodes. Notice that this implies that the first $\lfloor \log m \rfloor$ layers make up a complete binary tree structure, while all the remaining layers contain m nodes, for a total of $m - 1 + mn = s$ nodes. This hardcoded part of the graph will be used to force the isomorphism previously discussed. It is highly suggested to read the rest of this proof while looking at ??.

The idea is to map each block of $\text{Ref}_s(\varphi)$ to a node of G_α . In particular, the last block B_s is placed at the root of G_α , the node on layer 0, and the rest of the blocks are mapped to the graph decreasing order from left to right across all layers.

Next, we consider the block pointers given by the total assignment α . For each of these pointers, we'll add an edge to G_α corresponding to it. For the first $\lfloor \log m \rfloor$ layers, we hardcode the edges in G_α in a way that they represent a full binary tree. Notice that this part of the pointer assignment doesn't depend on the variables α .

Now, for each of the remaining layers $\ell \in [n] \setminus [\log m]$, we think of the $2m$ pointers going from the m blocks of layer ℓ to those of layer $\ell + 1$ as the $2m$ pigeons going to the m holes. More precisely, we define the left pointer of the i -th block on layer ℓ as the $h(2i - 1)$ -th block on layer $\ell + 1$. Similarly, the right pointer is the $h(2i)$ -th block on layer $\ell + 1$. When $h(i)$ is undefined, which can happen only when an axiom of rPHP_m^{2m} associated with i is violated, we define the corresponding pointer as *null* and we point it directly to the root block B_s , which automatically results in an axiom violation for $\text{Ref}_s(\varphi)$.

We observe that the resulting topology (ignoring null pointers) is a forest of binary trees: it is constructed by stitching together a binary tree at the top with a layered sequence of copies of the same partial matchings where we have identified pairs of pigeons. Now that the topology of G_α has been defined, we'll show that one of its subtrees is isomorphic to $T_\alpha \subseteq T_n$.

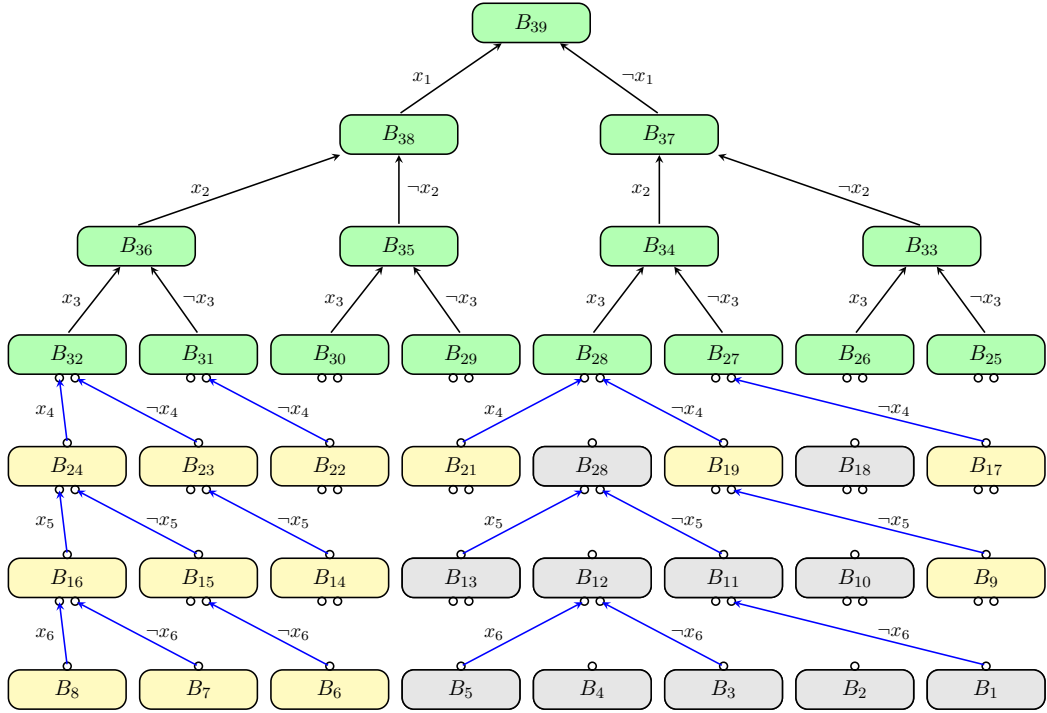


Figure 7. Reduction from rPHP_m^{2m} to $\text{Ref}_{m(n+1)-1}(\varphi)$ with $n = 7$ and $m = 8$. The copies of the partial matching defined by $h : [2m] \rightarrow [m]$ are represented by blue arrows. Hardcoded nodes are represented in green. Derived nodes that survive the walk-up phase are represented in yellow, while disabled blocks are represented in gray. The subtree isomorphic to T_α is given by green and yellow nodes.

Consider the following procedure to construct a decision tree that computes the literal set for a block B . Starting from B , we walk up G_α to its unique parent in the binary forest by computing the inverse of h using $O(\log m)$. Then, we repeat this process walking upwards for at most n steps until we reach a block B^* without a parent. We have two cases:

- $B^* = B_s$. Consider the reverse path p from B^* to B , viewed as a sequence of left and right turns. The path p identifies a node v of T_α obtained by following the same sequence of turns described by p . Once v is found, we copy all the local structure of v into B . In particular, the literal set of B is made equal to that of v , where a left turn at layer ℓ is associated with x_ℓ and a right turn is associated with $\neg x_\ell$. If v is derived in T_α by resolving on x_k , we make B a derived block and set its resolved variable to x_k . Similarly, if v is a leaf of T_α that is a weakening of some axiom $C_A \in \varphi$, we make B an axiom block and set its weakening index to A .
- $B^* \neq B_s$. In this case, the block B is set as disabled since it is not part of the subtree isomorphic to T_α . All of the other variables of B are randomly assigned since they're not needed.

Through this procedure, the whole contents of a particular block can be computed by a single decision tree of depth $O(n \log m)$.

To conclude the proof it remains to show that every axiom in $\text{Ref}_s(\varphi)$ (composed with the reduction) is implied by some axiom of rPHP_m^{2m} . We argue the contrapositive statement. Since the reduction produces a purported refutation isomorphic to a subtree $T_\alpha \subseteq T$ together with some disabled blocks which do not violate axioms of $\text{Ref}_{m(n+1)-1}(\varphi)$, the only possible axiom violations in $\text{Ref}_{m(n+1)-1}(\varphi)$ are caused by a block on a non-hardcoded layer $\ell \in [n] \setminus [\log m]$ containing a null pointer. Any null pointer is caused by the decision tree querying a pigeon i with $h(i) = *$, which means that the decision tree has witnessed an axiom violation of the form $f(i) = j \rightarrow g(j) = i$ for rPHP_m^{2m} . $\square$

Lemma 4.6. *Let φ be an unsatisfiable CNF formula with n variables and m clauses. Then, for all $s \in \mathbb{N}$ with $s \geq n^2$, the formula $\text{Ref}_s(\varphi)$ requires Resolution refutations of block-width $\Omega\left(\frac{s}{n^2 \log(s/n)}\right)$.*

Proof. Set $m = (s - 1)/(n + 1)$. By ?? we know that $\text{rPHP}_m^{2m} \stackrel{dt}{\leq}_{O(n \log m)} \text{Ref}_s(\varphi)$ via a block-aware reduction. Therefore, by ?? and ?? we conclude that:

$$\text{bw}_{\text{Res}}(\text{Ref}_s(\varphi)) \geq \Omega\left(\frac{\text{width}_{\text{Res}}(\text{rPHP}_m^{2m})}{n \log m}\right) \geq \Omega\left(\frac{s}{n^2 \log(s/n)}\right)$$

$\square$

4.3.4 Tradeoffs between block-width and size

4.4 Extending non-automatability up to Cutting Planes

4.5 Automatability and Feasible interpolation

4.5.1 Non-automatability of AC^0 -Frege

Chapter 5

Analyzability (TODO)

5.1 Do proofs of $\neg\text{Ref}$ leak information?

5.2 Analyzability of Resolution

5.2.1 Reducing block-width in Ref proofs

5.2.2 Extracting assignments from narrow Ref proofs

5.3 Extending analyzability up to PCR

5.4 Analyzability and feasible interpolation

Conclusions

Acknowledgements

Bibliography

- [Pou92] William Poundstone. *Prisoner's Dilemma: John Von Neumann, Game Theory and the Puzzle of the Bomb*. Doubleday, 1992. ISBN: 978-0385415804.
- [Sip92] Michael Sipser. “The history and status of the P versus NP question”. In: *Proceedings of the Twenty-Fourth Annual ACM Symposium on Theory of Computing*. STOC '92. Association for Computing Machinery, 1992. DOI: 10.1145/129712.129771.
- [Tur37] A. M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society* s2-42.1 (Jan. 1937), pp. 230–265. ISSN: 0024-6115. DOI: 10.1112/plms/s2-42.1.230.